

1

Lab

PHỤC VỤ MỤC ĐÍCH GIÁO DỤC
FOR EDUCATIONAL PURPOSE ONLY

ÔN TẬP NGÔN NGỮ ASSEMBLY & CHÈN MÃ VÀO TẬP TIN PE

Thực hành môn Cơ chế hoạt động của mã độc

Tháng 9/2025

Lưu hành nội bộ

<Nghiêm cấm đăng tải trên internet dưới mọi hình thức>

ÔN TẬP NGÔN NGỮ ASSEMBLY

A. TỔNG QUAN

1. Mục tiêu

- Tìm hiểu ngôn ngữ assembly (hợp ngữ)
- Thực hiện một số chương trình mẫu trên IDE online

2. Kiến thức nền tảng

Hợp ngữ là một ngôn ngữ lập trình cấp thấp đối với một máy tính hoặc thiết bị lập trình khác đặc trưng cho một kiến trúc máy tính cụ thể trái ngược với hầu hết các ngôn ngữ lập trình cấp cao, nói chung là nhiều hệ thống trên di động. Hợp ngữ được chuyển thành mã máy thực thi một số trình biên dịch như NASM, MASM...

Một ví dụ về hợp ngữ chương trình xuất ra chữ "Hello, world":

```
section    .text

    global _start    ;must be declared for linker (ld)
_start:                ;tells linker entry point

    mov     edx,len    ;message length
    mov     ecx,msg    ;message to write
    mov     ebx,1      ;file descriptor (stdout)
    mov     eax,4      ;system call number (sys_write)
    int     0x80        ;call kernel

    mov     eax,1      ;system call number (sys_exit)
    int     0x80        ;call kernel

section    .data
msg db 'Hello, world!', 0xa ;our dear string
len equ $ - msg          ;length of our dear string
```

Hợp ngữ là gì?

Mỗi máy tính cá nhân đều có một bộ vi xử lý (CPU), đảm nhiệm các chức năng số học, logic và điều khiển hoạt động của hệ thống.

Mỗi dòng vi xử lý được thiết kế với một tập lệnh (instruction set) riêng để thực hiện các thao tác khác nhau, ví dụ như nhận dữ liệu từ bàn phím, hiển thị thông tin lên màn hình, hoặc điều khiển các thiết bị ngoại vi. Các tập lệnh này chính là ngôn ngữ máy, được biểu diễn trực tiếp bằng các chuỗi nhị phân (0 và 1) mà bộ xử lý có thể hiểu và thực thi.

Tuy nhiên, ngôn ngữ máy có cú pháp phức tạp, khó ghi nhớ và khó áp dụng trong phát triển phần mềm. Để khắc phục hạn chế này, người ta xây dựng ngôn ngữ hợp ngữ (assembly language) – một ngôn ngữ lập trình bậc thấp, sử dụng ký hiệu mnemonics thay thế cho các chuỗi nhị phân. Hợp ngữ được thiết kế gắn liền với một kiến trúc vi xử lý cụ thể, giúp con người có thể lập trình ở mức gần với phần cứng nhưng dễ hiểu và dễ quản lý hơn so với ngôn ngữ máy.

Ưu điểm của hợp ngữ

Có một sự hiểu biết về ngôn ngữ lắp ráp làm cho người ta nhận thức được:

- Làm thế nào các chương trình giao diện với các hệ điều hành, bộ xử lý, và BIOS;
- Làm thế nào dữ liệu được đại diện trong bộ nhớ và các thiết bị bên ngoài khác;
- Làm thế nào các bộ vi xử lý truy cập và thực hiện các hướng dẫn;
- Làm thế nào hướng dẫn truy cập và xử lý dữ liệu;
- Làm thế nào một chương trình truy cập các thiết bị bên ngoài.

Những lợi ích khác của việc sử dụng ngôn ngữ lắp ráp là:

Nó đòi hỏi ít bộ nhớ và thời gian thực hiện;

- Nó cho phép phần cứng cụ thể công việc phức tạp một cách dễ dàng hơn;
- Nó phù hợp cho công việc thời gian quan trọng;
- Nó phù hợp nhất để viết các thủ tục dịch vụ ngắt và chương trình thường trú bộ nhớ khác.

Các tính năng cơ bản của phần cứng máy

Các phần cứng nội bộ chính của một máy tính bao gồm bộ vi xử lý, bộ nhớ, và đăng ký. Đăng ký là thành phần xử lý mà giữ dữ liệu và địa chỉ. Để thực hiện một chương trình, các bản sao hệ thống từ các thiết bị bên ngoài vào bộ nhớ trong. Các bộ vi xử lý thực hiện các hướng dẫn của chương trình.

Các đơn vị cơ bản của lưu trữ máy tính là một chút; nó có thể là ON (1) hoặc OFF (0). Một nhóm chín bit liên quan làm cho một byte, trong đó tám bit được sử dụng cho dữ liệu và là người cuối cùng được sử dụng cho tính chẵn lẻ. Theo quy luật chẵn lẻ, số lượng bit được ON (1) trong mỗi byte nên luôn luôn là số lẻ.

Vì vậy, các bit chẵn lẻ được sử dụng để làm cho số bit trong một byte lẻ. Nếu tính chẵn lẻ là chẵn, hệ thống giả định rằng đã có một lỗi chẵn lẻ (dù hiếm), mà có thể đã được gây ra do lỗi phần cứng hoặc xáo trộn điện.

Các bộ xử lý hỗ trợ các kích thước dữ liệu sau:

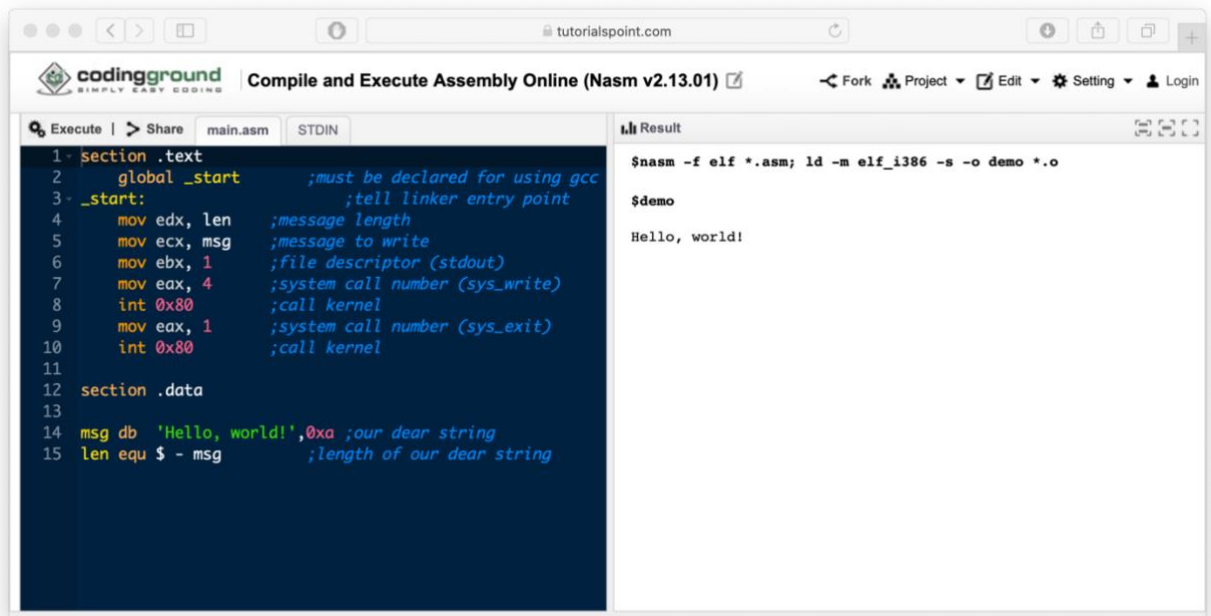
- Word: một mục dữ liệu 2-byte
- Doubleword: a 4-byte (32 bit) mục dữ liệu
- Quadword: một byte 8 (64 bit) mục dữ liệu
- Đoạn: 16-byte (128 bit) khu vực
- Kilobyte: 1024 bytes
- Megabyte: 1.048.576 byte

B. CHUẨN BỊ MÔI TRƯỜNG

**Lựa chọn một trong hai cách sau*

1. Chạy code assembly trên IDE online

1. Truy cập http://www.tutorialspoint.com/compile_assembly_online.php
2. Nhấn vào Excute để chạy tập tin thực thi:



2. Chạy code assembly trên Linux NASM

1. Tạo tập tin nguồn *test.asm*

```

khoanh@khoanh-inseclab:~$ cat test.asm
section .text
    global _start ;must be declared for using gcc
_start: ;tell linker entry point
    mov     edx, len ;message length
    mov     ecx, msg ;message to write
    mov     ebx, 1 ;file descriptor (stdout)
    mov     eax, 4 ;system call number (sys_write)
    int     0x80 ;call kernel
    mov     eax, 1 ;system call number (sys_exit)
    int     0x80 ;call kernel

section .data

msg     db     'Hello, world!',0xa ;our dear string
len     equ     $ - msg ;length of our dear string
  
```

2. Chạy lệnh **nasm -f elf test.asm** để biên dịch code ra tập tin *test.o*

3. Tạo tập tin thực thi **ld -m elf_i386 test.o -o test**

4. Chạy tập tin thực thi **./test**

C. THỰC HÀNH

1. Cấu trúc cơ bản của một chương trình

```
1 section .text
2     global _start           ;must be declared for using gcc
3     _start:                 ;tell linker entry point
4     mov edx, len            ;message length
5     mov ecx, msg            ;message to write
6     mov ebx, 1              ;file descriptor (stdout)
7     mov eax, 4              ;system call number (sys_write)
8     int 0x80                ;call kernel
9     mov eax, 1              ;system call number (sys_exit)
10    int 0x80                ;call kernel
11
12 section .data
13
14 msg db 'Cau truc co ban cua mot chuong trinh',0xa ;our dear string
15 len equ $ - msg           ;length of our dear string
16
```

1. Phần **.text** là phần chính chạy chương trình.
2. Phần **.data** là phần khai báo biến và hằng của chương trình ở đây chúng ta khai báo hai biến `msg` và `len`.
3. Gõ và thực thi chương trình trên IDE Online hoặc NASM và quan sát kết quả trả về.

```
khoanh@khoanh-inseclab:~$ nasm -f elf *.asm; ld -m elf_i386 -s -o demo *.o
khoanh@khoanh-inseclab:~$ ./demo
Cau truc co ban cua mot chuong trinh
khoanh@khoanh-inseclab:~$
```

2. Địa chỉ thanh ghi, khai báo biến và gọi hàm hệ thống

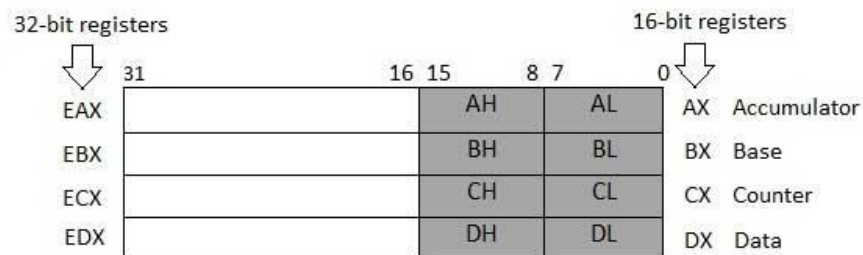
- Thực thi chương trình sau và quan sát kết quả.

```

1 section .text
2     global _start      ;must be declared for linker (gcc)
3
4 _start:                ;tell linker entry point
5     mov     edx,len     ;message length
6     mov     ecx,msg     ;message to write
7     mov     ebx,1       ;file descriptor (stdout)
8     mov     eax,4       ;system call number (sys_write)
9     int     0x80        ;call kernel
10
11     mov     edx,5       ;message length
12     mov     ecx,s2      ;message to write
13     mov     ebx,1       ;file descriptor (stdout)
14     mov     eax,4       ;system call number (sys_write)
15     int     0x80        ;call kernel
16
17     mov     eax,1       ;system call number (sys_exit)
18     int     0x80        ;call kernel
19
20 section .data
21 msg db 'Displaying 5 stars', 0xa ;a message
22 len equ $ - msg        ;length of message
23 s2 times 5 db '*'

```

- Trong ví dụ sử dụng các vùng địa chỉ trên thanh ghi như sau: **edx, ecx, ebx, eax** để lưu các giá trị sau đó gọi hàm hệ thống tương ứng.



- Các hàm hệ thống được gọi: `sys_write(4)`, `sys_exit(1)`.

%eax	Name	%ebx	%ecx	%edx	%esx	%edi
1	sys_exit	int	-	-	-	-
2	sys_fork	struct pt_regs	-	-	-	-
3	sys_read	unsigned int	char *	size_t	-	-
4	sys_write	unsigned int	const char *	size_t	-	-
5	sys_open	const char *	int	int	-	-
6	sys_close	unsigned int	-	-	-	-

4. Các kiểu biến sử dụng trong assembly.

Directive	Purpose	Storage Space
DB	Define Byte	allocates 1 byte
DW	Define Word	allocates 2 bytes
DD	Define Doubleword	allocates 4 bytes
DQ	Define Quadword	allocates 8 bytes
DT	Define Ten Bytes	allocates 10 bytes

Quan sát dòng **code 22**, ta có dòng khai báo **equ** đây là cách **khai báo hằng** trong assembly. Dấu \$ trong trường hợp này là vùng địa chỉ(byte) ngay sau vùng địa chỉ khai báo biến **msg**.

3. Instruction

1. Hiện thực chương trình và quan sát kết quả.

```

SYS_EXIT equ 1
SYS_READ equ 3
SYS_WRITE equ 4
STDIN equ 0
STDOUT equ 1

segment .data

msg1 db "Enter a digit ", 0xA,0xD
len1 equ $- msg1

msg2 db "Please enter a second digit", 0xA,0xD
len2 equ $- msg2

msg3 db "The sum is: "
len3 equ $- msg3

```



```
msg4 db 0xA
len4 equ $- msg4

segment .bss

num1 resb 2
num2 resb 2
res resb 1

section .text
global _start ;must be declared for using gcc

_start: ;tell linker entry point
mov eax, SYS_WRITE
mov ebx, STDOUT
mov ecx, msg1
mov edx, len1
int 0x80

mov eax, SYS_READ
mov ebx, STDIN
mov ecx, num1
mov edx, 2
int 0x80

mov eax, SYS_WRITE
mov ebx, STDOUT
mov ecx, msg2
mov edx, len2
int 0x80

mov eax, SYS_READ
mov ebx, STDIN
mov ecx, num2
mov edx, 2
int 0x80

mov eax, SYS_WRITE
mov ebx, STDOUT
mov ecx, msg3
mov edx, len3
int 0x80

; moving the first number to eax register and second number to ebx
; and subtracting ascii '0' to convert it into a decimal number

mov eax, [num1]
sub eax, '0'

mov ebx, [num2]
sub ebx, '0'
```

```
; add eax and ebx
add eax, ebx
; add '0' to to convert the sum from decimal to ASCII
add eax, '0'

; storing the sum in memory location res
mov [res], eax

; print the sum
mov eax, SYS_WRITE
mov ebx, STDOUT
mov ecx, res
mov edx, 1
int 0x80

; new line
mov eax, SYS_WRITE
mov ebx, STDOUT
mov ecx, msg4
mov edx, len4
int 0x80

exit:

mov eax, SYS_EXIT
xor ebx, ebx
int 0x80
```

2. Chương trình sử dụng lệnh (instruction) add để cộng hai giá trị ở vùng địa chỉ: **eax** và **ebx** lại, sau đó chuyển đổi từ số(decimal) sang chuỗi ASCII bằng cách cộng thêm kí tự '0'

```
; add eax and ebx
add eax, ebx
; add '0' to to convert the sum from decimal to ASCII
add eax, '0'
```

Ngoài ra còn có một số lệnh(instruction) như sau:

- **SUB**: trừ hai vùng địa chỉ
- **MUL**: nhân hai vùng địa chỉ
- **IMUL**: chia hai vùng địa chỉ
- **AND**: phép logic AND giữa 2 vùng địa chỉ
- **OR**: Phép logic OR giữa 2 vùng địa chỉ
- **XOR**: Phép logic XOR giữa 2 vùng địa chỉ



- **TEST:** Phép AND có biến nhớ(flag)
 - **NOT:** Phép phủ định
3. **MOV:** Phép gán dữ liệu từ một vùng này sang một vùng khác hoặc giá trị vào một vùng địa chỉ.

```
MOV register, register
MOV register, immediate
MOV memory, immediate
MOV register, memory
MOV memory, register
```

4. Điều kiện rẽ nhánh (Condition)

1. Hiện thực chương trình sau và quan sát kết quả.

```
section .text
global _start ;must be declared for using gcc

_start: ;tell linker entry point
    mov ecx, [num1]
    cmp ecx, [num2]
    jg check_third_num
    mov ecx, [num2]

check_third_num:

    cmp ecx, [num3]
    jg _exit
    mov ecx, [num3]

_exit:

    mov [largest], ecx
    mov ecx, msg
    mov edx, len
    mov ebx, 1 ;file descriptor (stdout)
    mov eax, 4 ;system call number (sys_write)
    int 0x80 ;call kernel

    mov ecx, largest
    mov edx, 2
    mov ebx, 1 ;file descriptor (stdout)
    mov eax, 4 ;system call number (sys_write)
    int 0x80 ;call kernel

    mov eax, 1
    int 80h
```

```
section .data

msg db "The largest digit is: ", 0xA,0xD
len equ $- msg
num1 dd '7'
num2 dd '2'
num3 dd '1'

segment .bss
largest resb 2
```

2. Chương trình trên tìm ra giá trị lớn nhất trong 3 số truyền vào (num1, num2, num3). Trong chương trình này sử dụng lệnh *jump* có điều kiện kết hợp với lệnh *cmp*:

```
jg check_third_num
```

- **jg**(Jump Greater) : lệnh jump được thực thi khi giá trị so sánh của lệnh *cmp* lớn hơn.
 - **check_third_num**: là một nhãn địa chỉ để lệnh *jump* hướng tới thực thi.
3. Ban đầu chương trình thực hiện so sánh hai số num1 và num2.

```
mov ecx, [num1]
cmp ecx, [num2]
jg check_third_num
mov ecx, [num2]
```

4. Sau đó **jg** được thực thi để so sánh với num3.

```
check_third_num:

cmp ecx, [num3]
jg _exit
mov ecx, [num3]
```

Lúc này ecx sẽ đang là num1, ta so sánh tiếp với num3 thông qua lệnh: **cmp ecx, [num3]**.

Nếu **ecx(num1)** lớn hơn **num3** thì ta **_exit**, tức là **ecx(num1)** là giá trị lớn nhất được tìm thấy.

Trong trường hợp **num1** nhỏ hơn **num2** thì lệnh **check_third_num** sẽ không được thực thi, thay vào đó chương trình tiếp tục gán **num2** cho **ecx** để tiếp tục so sánh, nhãn **check_third_num** được thực thi.

Nếu **ecx** lớn hơn **num3** thì jg được thực thi tới nhãn **_exit**, tức là ecx(num2) là giá trị lớn nhất được tìm thấy => dừng chương trình. Ngược lại, num3 được gán cho ecx, chương trình tiếp tục thực thi tới nhãn **_exit**. Lúc này ecx(num3) sẽ được xem là giá trị lớn nhất được tìm thấy.

Bài thực hành 1: Tìm số nhỏ nhất trong một danh sách số bất kỳ

Viết chương trình tìm **số nhỏ nhất** trong **một mảng** gồm **N số nguyên** (giá trị có thể lớn hoặc nhỏ hơn 1 chữ số).

Yêu cầu:

- Sử dụng **mảng** để lưu danh sách số nguyên thay vì khai báo từng số riêng lẻ.
- Dùng **vòng lặp** để duyệt qua các phần tử thay vì so sánh từng số một.
- Lưu kết quả vào biến **min_result**.

🔍 Dừng lại và suy nghĩ:

- 1.1. Làm thế nào để tìm số lớn nhất thay vì số nhỏ nhất?
- 1.2. Nếu danh sách có số lượng phần tử rất lớn (hàng triệu số), làm thế nào để tối ưu hiệu suất?

🔗 Hoàn thành đoạn mã:

```
section .data
    numbers dd ?, ?, ?, ? ; Array containing N integers
    n dd ? ; Number of elements in the array
    min_result dd 0 ; Result

section .text
    global _start

_start:
    ; Write code to iterate through the array to find the smallest number here

    ; Store the result in min_result

    ; Exit the program
```

Bài thực hành 2: Chuyển đổi số nguyên thành chuỗi HEX và in ra màn hình

Viết chương trình để **chuyển số nguyên thành chuỗi HEX** và in ra màn hình.

Yêu cầu:

- Lấy một số nguyên (ví dụ 305419896) và chuyển nó thành dạng **chuỗi HEX** (0x12345678).
- Sử dụng **phép toán dịch bit** thay vì phép chia thông thường để tối ưu hiệu suất.
- Hiển thị kết quả ra màn hình bằng syscall.

Gợi ý:

- Sử dụng phép toán AND (AND eax, 0xF) để trích từng chữ số HEX.
- Chuyển đổi sang ký tự ASCII ('0' - '9' hoặc 'A' - 'F').
- Ghi chuỗi kết quả vào bộ nhớ rồi in ra màn hình.

 Dừng lại và suy nghĩ:

2.1. Tại sao sử dụng dịch bit (SHR) lại nhanh hơn phép chia?

2.2 Làm thế nào để mở rộng chương trình này thành chuyển đổi số bất kỳ (có dấu) thành HEX?

 Hoàn thành đoạn mã:

```
section .bss
    buffer resb 10 ; Memory to store the HEX string

section .data
    number dd ? ; Number to be converted
    prefix db "0x", 0 ; Prefix for HEX number
    newline db 10 ; Newline character

section .text
    global _start

_start:
    ; Write code to convert the number to HEX here

    ; Print to the screen

    ; Exit the program
```

Bài thực hành 3: Phát hiện số đối xứng (Palindrome) trong Assembly

Viết chương trình kiểm tra xem một số nguyên có phải là **số đối xứng (Palindrome Number)** hay không.

Yêu cầu:

- Nhập một số nguyên N.
- Kiểm tra xem khi **đảo ngược các chữ số**, số đó có giống số ban đầu không.
- Hiển thị kết quả "YES" nếu là số đối xứng, "NO" nếu không.

Gợi ý:

- Sử dụng phép chia để **tách từng chữ số**.
- Sử dụng một biến tạm để **tạo số đảo ngược**.
- So sánh với số gốc để quyết định kết quả.

 Dừng lại và suy nghĩ:

- 3.1. Làm thế nào để tối ưu thuật toán kiểm tra số đối xứng mà không cần biến trung gian?
- 3.2. Nếu nhập vào một số **rất lớn**, chương trình có gặp vấn đề gì không?

 Hoàn thành đoạn mã:

```
section .data
    number dd ? ; Number to be checked
    result_yes db "YES", 10
    result_no db "NO", 10

section .text
    global _start

_start:
    ; Write code to check if the number is a palindrome here

    ; Print the result to the screen

    ; Exit the program
```

Bài thực hành 4: XOR mã hóa chuỗi

Viết chương trình thực hiện **mã hóa XOR** một chuỗi bằng một khóa bí mật.

Yêu cầu:

- Nhập vào một chuỗi "HELLO WORLD".
- Mã hóa bằng **phép toán XOR** với một khóa (ví dụ: 0xAA).
- Hiển thị kết quả mã hóa lên màn hình.
- Giải mã lại để kiểm tra kết quả đúng.

Gợi ý:

- Duyệt từng ký tự trong chuỗi và thực hiện **XOR** với khóa.
- Dùng **vòng lặp** để xử lý toàn bộ chuỗi.
- Dùng syscall để in kết quả ra màn hình.

 Dừng lại và suy nghĩ:

- 4.1. Nếu **không biết khóa**, có thể giải mã chuỗi không?
- 4.2. Tại sao XOR là phương pháp phổ biến trong mã độc?

CHÈN MÃ VÀO TẬP TIN PE

D. TỔNG QUAN

1. Mục tiêu

- Hiểu được cơ chế chèn mã vào file PE của viruses.
- Thực hiện chèn một đoạn mã đơn giản để hiển thị một message box khi mở file NOTEPAD.exe

2. Chuẩn bị môi trường

- Cài đặt [CFF Explorer](#) để xem và chỉnh sửa các tham số trong file PE.
- Cài đặt [HxD](#) để đọc và sửa nội dung file PE.
- Cài đặt **IDA Pro** để kiểm tra mã hợp ngữ của chương trình muốn chèn.

(Vui lòng sử dụng máy ảo Windows để thực hành – liên hệ GVHD để lấy các công cụ liên quan)

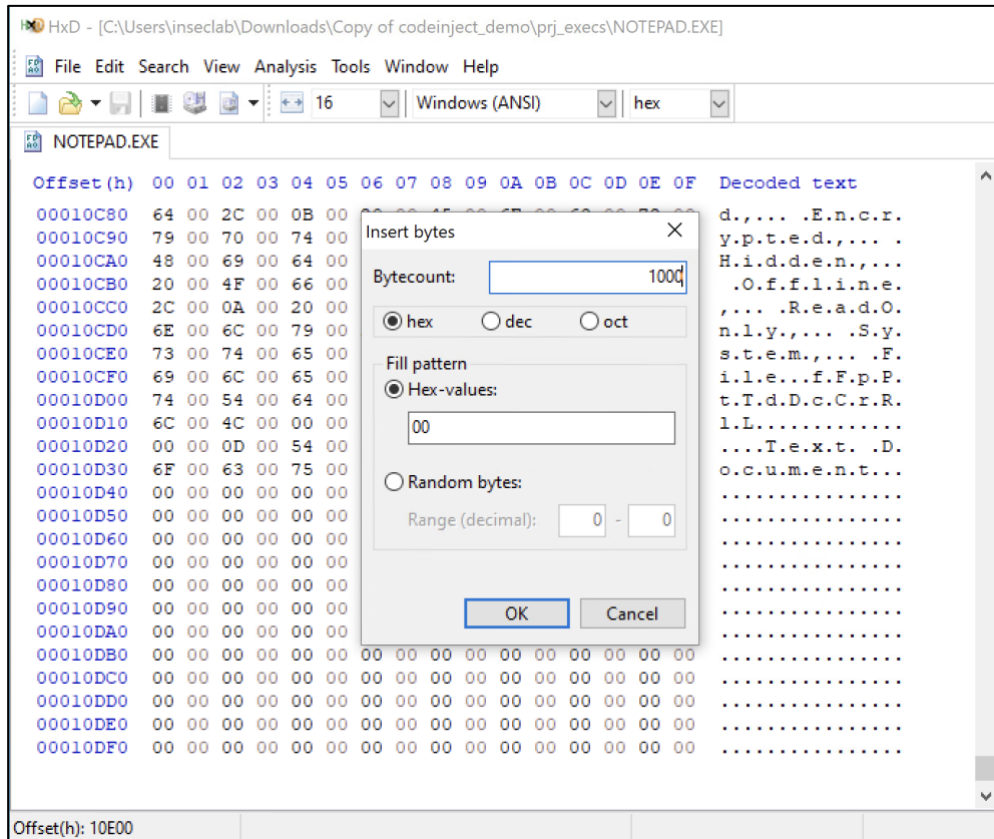
E. THỰC HÀNH

Mở CFF Explorer để đọc nội dung của Notepad.exe. Nhìn vào các nội dung được hiển thị.

- Chọn Optional Header, tìm **AddressOfEntryPoint** trong bảng bên phải chứa giá trị 0x0000739D. Cũng trong bảng bên phải, tìm **ImageBase** chứa giá trị 0x01000000. Cộng 2 giá trị này lại ta được 0x0100739D. Đây chính là địa chỉ ảo (**virtual address**) xác định vị trí thực thi của chương trình.
- Chọn Section Header, ta thấy 3 sections: text, data và rsrc. Để đơn giản, chúng ta sẽ mở rộng rsrc và chèn code vào đây. Mỗi section có chứa 4 thông tin:
 - VirtualSize: kích thước của section khi được load vào bộ nhớ.
 - VirtualAddress(VA): địa chỉ của section khi được load vào bộ nhớ.
 - RawSize: kích thước của section trong PE file.
 - RawAddress(RA): địa chỉ của section trong PE file.

1. Tạo vùng nhớ trong tập tin PE

Sử dụng HxD để mở *Notepad.exe*. **Đặt trỏ chuột vào cuối file**, chọn **Edit -> Insert Bytes**, nhập giá trị *0x1000* (có thể chèn nhiều hơn đối với các đoạn mã phức tạp, tuy nhiên sẽ làm tăng kích thước của PE file) và nhấn OK.



2. Tạo chương trình cần chèn

1. Ta có một chương trình hiển thị MessageBox như sau:

```
#include <windows.h>
int main(int argc, char * argv[])
{
    MessageBox(NULL, L"Info", L"Code injected", MB_OK);
    return 0;
}
```

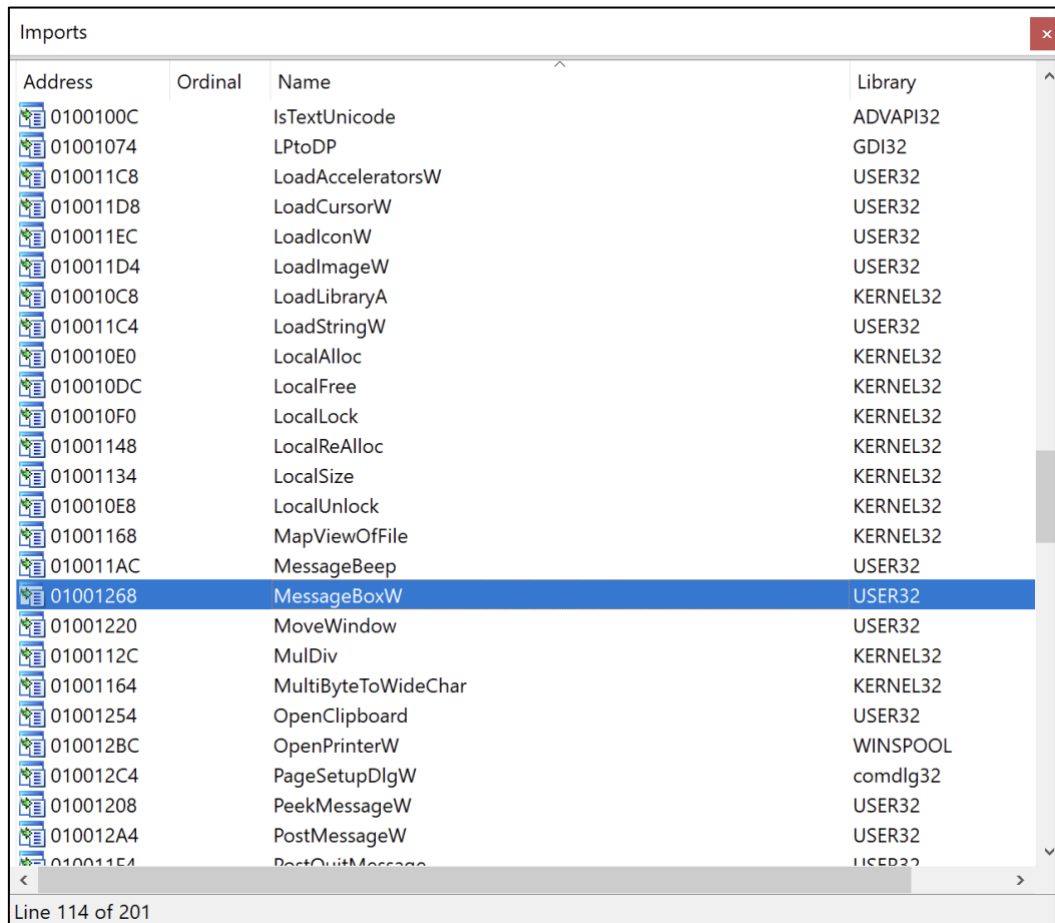
2. Biên dịch chương trình dưới chế độ *Release, Not Using Precompiled Headers*. Sử dụng IDA Pro để mở file PE và xem mã hợp ngữ của chương trình vừa biên dịch.

3. Về cơ bản, chương trình gồm 5 dòng lệnh (sử dụng chức năng hexview để xem mã hex của từng lệnh).

```
push 0 ; 6a 00
push Caption ; 68 X
push Text ; 68 Y
push 0 ; 6a 00
call [MessageBoxW] ; ff15 Z
```

Để chèn đoạn code này vào Notepad.exe, ta phải đi tìm các giá trị (X, Y, Z) phù hợp.

4. Giá trị **Z** chính là địa chỉ của hàm *MessageBoxW* được import từ thư viện USER32.dll. Trong IDA Pro, mở *Notepad.exe*, chọn **View -> Open Subviews -> Imports** và ta thấy địa chỉ của hàm *MessageBoxW* chính là 01001268.

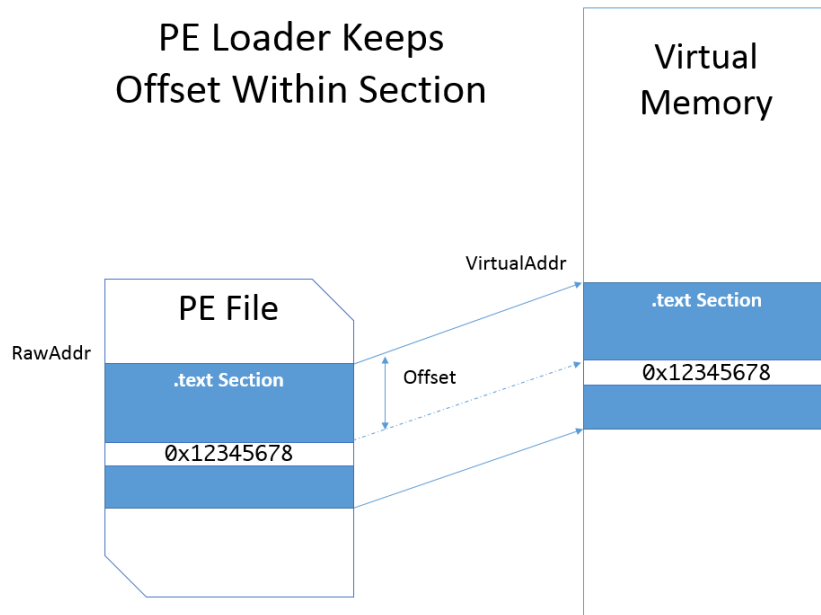


Address	Ordinal	Name	Library
0100100C		IsTextUnicode	ADVAPI32
01001074		LPTODP	GDI32
010011C8		LoadAcceleratorsW	USER32
010011D8		LoadCursorW	USER32
010011EC		LoadIconW	USER32
010011D4		LoadImageW	USER32
010010C8		LoadLibraryA	KERNEL32
010011C4		LoadStringW	USER32
010010E0		LocalAlloc	KERNEL32
010010DC		LocalFree	KERNEL32
010010F0		LocalLock	KERNEL32
01001148		LocalReAlloc	KERNEL32
01001134		LocalSize	KERNEL32
010010E8		LocalUnlock	KERNEL32
01001168		MapViewOfFile	KERNEL32
010011AC		MessageBeep	USER32
01001268		MessageBoxW	USER32
01001220		MoveWindow	USER32
0100112C		MulDiv	KERNEL32
01001164		MultiByteToWideChar	KERNEL32
01001254		OpenClipboard	USER32
010012BC		OpenPrinterW	WINSPOOL
010012C4		PageSetupDlgW	comdlg32
01001208		PeekMessageW	USER32
010012A4		PostMessageW	USER32
010011E4		PostQuitMessage	USER32

5. Trong HxD, ta chọn địa chỉ 0x00011000 trong vùng nhớ đã được mở rộng (bước C1) để lưu trữ mã hợp ngữ, 0x00011040 để lưu trữ Caption và 0x00011060 để lưu trữ Text. Ta có thể tùy chọn những vị trí khác tùy thích.

Đối với mỗi section, loader sẽ copy section tại RA trong PE file sang bộ nhớ ảo tại VA trong khi vẫn giữ đúng offset bên trong section đó.

$$\text{Offset} = \text{RA} - \text{Section RA} = \text{VA} - \text{Section VA} \quad (1)$$



Giá trị X có thể được tìm dựa vào công thức (1)

$$0x00011040 - 0x00008400 = X - 0x000B000$$

$$X = 0x00013C40$$

Cộng thêm ImageBase, suy ra $X = 0x01013C40$. Tương tự, $Y = 0x01013C60$.

Như vậy, đoạn code này thực hiện chức năng như mong đợi và có địa chỉ mới là:

$$\text{new_entry_point} = 0x00011000 - 0x00008400 + 0x000B000 = 0x00013C00$$

3. Thiết lập lệnh quay về AddressOfEntryPoint ban đầu

- Để chương trình *Notepad.exe* tiếp tục được thực thi sau khi đã chạy đoạn code trên, ta cần chèn dòng lệnh quay về *AddressOfEntryPoint* cũ ngay sau đoạn code ở bước 2. `jmp relative_VA`
- Đối với lệnh `jmp`, đích đến (`old_entry_point`) sẽ được tính bằng cách cộng giá trị `relative_VA` vào thanh ghi PC khi lệnh được thực thi. Bởi vì PC luôn trở đến vị trí đầu của câu lệnh kế tiếp, cho nên cần phải tính 5 bytes của câu lệnh `jmp` nữa. Ta có công thức sau:

$$\text{old_entry_point} = \text{jmp_instruction_VA} + 5 + \text{relative_VA} \quad (2)$$

Nếu đặt lệnh `jmp` sau 5 câu lệnh ở bước 2 thì `jmp_instruction_VA = 0x01013C14`.

old_entry_point = 0x0100739D chính là giá trị AddressOfEntryPoint ban đầu đã cộng ImageBase.

Suy ra, relative_VA = 0x0100739D - 5 - 0x01013C14 = 0xFFFF3784.

- Đến đây, ta đã có một đoạn mã hợp ngữ hoàn chỉnh để chèn vào Notepad.exe. Các địa chỉ được biểu diễn theo thứ tự little endian (x86).

```
push 0          ; 6a 00
push Caption    ; 68 403C0101
push Text       ; 68 603C0101
push 0          ; 6a 00
call [MessageBoxW] ; ff 15 68120001
jmp Originl_Entry_Point ; e9 8437FFFF
```

4. Chèn vào Notepad.exe

Sử dụng HxD để chèn đoạn mã cùng với giá trị Caption và Text vào Notepad.exe. Lưu lại file.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00010FF0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011000	6A	00	68	60	3C	01	01	68	40	3C	01	01	6A	00	FF	15	j.h`<...hê<...j.ý.
00011010	68	12	00	01	E9	84	37	FF	FF	00	00	00	00	00	00	00	h...é„7ýý.....
00011020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011040	43	00	6F	00	64	00	65	00	20	00	69	00	6E	00	6A	00	C.o.d.e. .i.n.j.
00011050	65	00	63	00	74	00	65	00	64	00	00	00	00	00	00	00	e.c.t.e.d.....
00011060	49	00	6E	00	66	00	6F	00	00	00	00	00	00	00	00	00	I.n.f.o...0.....
00011070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00011090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000110A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

5. Hiệu chỉnh các tham số trong PE header

- Sử dụng CFF Explorer để thay đổi các giá trị sau:

- Trong Section Headers, thay đổi .rsrc Section Header.

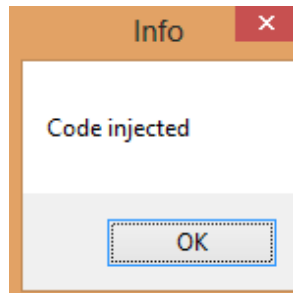
Name	Virtual Size	Virtual Address	Raw Size	Raw Address
Byte[8]	Dword	Dword	Dword	Dword
.text	00007748	00001000	00007800	00000400
.data	00001BA8	00009000	00000800	00007C00
.rsrc	00009958	0000B000	00009A00	00008400

- Trong Optional Headers, tăng SizeOfImage lên 0x1000.
- Trong Optional Headers, chỉnh sửa AddressOfEntryPoint thành 0x00013C00.

2. Lưu lại file.

6. Kiểm tra kết quả

Khi thực thi Notepad.exe, một cửa sổ xuất hiện như sau:



Bài thực hành 5: Thực hiện lại các bước trên thay đổi phần Text là MSSV.

Bài thực hành 6: Bằng cách không tạo thêm vùng nhớ mở rộng vào tập tin PE, tận dụng vùng nhớ trống để chèn chương trình cần chèn trên tập tin Notepad và calc.

7. Bài tập nâng cao

Bài thực hành 7: Chỉnh sửa luồng thực thi mà không thay đổi Entry Point.

Thay vì thay đổi trực tiếp **AddressOfEntryPoint**, bạn phải:

- **Xác định một function có sẵn** trong Notepad.exe mà được gọi sớm (ví dụ WinMain hoặc một hàm khởi tạo).
- **Chèn đoạn code mong muốn vào giữa một function hợp lệ** bằng cách thay đổi một số lệnh **không quan trọng** để chuyển hướng thực thi.
- Đảm bảo **trả lại quyền điều khiển** cho chương trình gốc sau khi payload đã chạy.

Gợi ý:

- Dùng **IDA Pro** để phân tích mã Assembly của Notepad.exe và tìm function phù hợp.
- **Thay thế một chuỗi lệnh không quan trọng** bằng jmp hoặc call đến shellcode.
- Tránh thay đổi **bảng nhập xuất API** để giảm nguy cơ phát hiện.

Bài thực hành 8: Hook API WriteFile hoặc ReadFile.**Yêu cầu:**

- Xác định **API WriteFile** hoặc **ReadFile** trong Notepad.exe.
- **Chèn một đoạn mã hook** để intercept các lệnh ghi file trước khi tiếp tục thực thi.
- **Ghi log dữ liệu** được ghi vào file trước khi cho phép Notepad tiếp tục.

Gợi ý:

- Dùng **IAT Hooking**: thay đổi địa chỉ của WriteFile trong bảng IAT thành shellcode.
- Dùng **Inline Hook**: ghi đè 5 byte đầu của WriteFile bằng một jmp đến shellcode.
- Đảm bảo **khôi phục lại hook** để Notepad không bị crash.

Bài thực hành 9: Mã hóa Shellcode bằng XOR.

Thay vì **chèn shellcode trực tiếp**, bạn phải:

- **Mã hóa shellcode bằng thuật toán XOR** với một khóa đơn giản (ví dụ 0x55).
- **Chỉ chèn shellcode đã mã hóa vào Notepad.exe.**
- **Viết một đoạn code nhỏ để giải mã shellcode** trước khi thực thi.

Gợi ý:

- **Duyệt qua từng byte của shellcode và XOR với 0x55** trước khi lưu vào PE.
- Dùng một **stub nhỏ để giải mã shellcode ngay trong bộ nhớ.**
- Không lưu khóa XOR dưới dạng **hằng số trong mã nguồn** để tránh bị phát hiện.

Bài thực hành 10: Che giấu dấu vết trong Header PE.

Để tránh bị phát hiện, yêu cầu:

- **Chỉnh sửa các giá trị trong PE Header** để làm cho file trông hợp lệ hơn.
- **Thay đổi TimeDateStamp, SizeOfImage, hoặc Section Names** để mô phỏng một tập tin hợp pháp.
- **Cập nhật lại checksum của tập tin PE** sau khi chỉnh sửa.

Gợi ý:

- Dùng **CFF Explorer** để thay đổi TimeDateStamp về một giá trị ngẫu nhiên.

- Giả mạo một tập tin PE hợp lệ bằng cách đổi tên các Section .text → .xyz1.
- Tăng nhẹ kích thước tập tin để tránh bị phát hiện bởi hash-based detection.

Bài thực hành 11: Mã tự thay đổi (Self-Modifying Code).

Yêu cầu:

- Viết shellcode có thể tự sửa đổi chính nó trong bộ nhớ trước khi chạy.
- Thực hiện kỹ thuật “polymorphic shellcode” để thay đổi opcode tại runtime.
- Không sử dụng jmp/call trực tiếp, mà thay vào đó sử dụng **stack pivoting**.

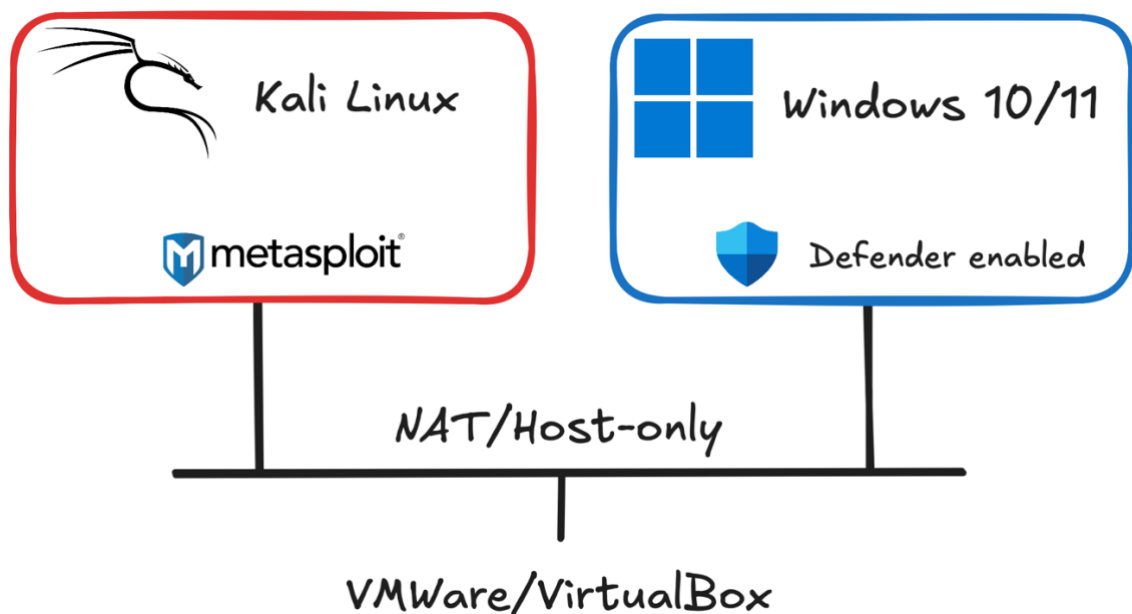
Gợi ý:

- Dùng **VirtualProtect** hoặc **VirtualAlloc** để làm shellcode có thể ghi.
- Tạo một vòng lặp đơn giản để thay đổi chính opcode của shellcode trước khi thực thi.
- Tận dụng “Return-Oriented Programming” (ROP) gadgets để tránh dùng trực tiếp jmp/call.

Bài thực hành 12: Tấn công khai thác lỗ hổng trong môi trường mạng nội bộ.

✂ Yêu cầu:

Sinh viên sẽ triển khai một mô hình mạng nội bộ **mô phỏng kịch bản tấn công thực tế**, sử dụng kỹ thuật **shellcode injection** và **persistence** để duy trì quyền truy cập từ xa vào hệ thống mục tiêu.



Mô hình hệ thống

- **Máy A:** Cài đặt **Metasploit Framework** (Kali Linux, Parrot OS hoặc Windows có Metasploit).
- **Máy B:** Chạy **Windows 10/11 (bản cập nhật mới nhất)** với **Windows Defender** được bật.
- **Mạng:** Máy A và B **cùng trong mạng nội bộ** (có thể sử dụng **VMware** hoặc **VirtualBox** để tạo mạng ảo nội bộ).

Nhiệm vụ:

10.1. Reverse Shell Payload Execution

- Từ máy A, sử dụng **msfvenom** để tạo **Reverse HTTP Shellcode**.
- **Viết chương trình C/C++ trên máy B** để thực thi shellcode này, sao cho khi chạy chương trình, **máy B sẽ kết nối ngược (reverse shell) về máy A**.
- **Thiết lập persistence:** Tìm cách **tự động thực thi chương trình** khi máy B khởi động lại. **Tối thiểu 3 cách**.

Dừng lại và suy nghĩ:

- Nếu Windows Defender phát hiện shellcode, làm sao để bypass?
- Làm thế nào để thực thi shellcode mà không lưu file .exe trên ổ đĩa?

10.2 Shellcode Injection vào Ứng Dụng Hợp Pháp (WinRAR)

- Từ máy A, tạo **Reverse Shell Payload** giống như bài 10.1.
- **Tìm cách chèn shellcode vào chương trình WinRAR** trên máy B.
- **Khi người dùng mở WinRAR, shellcode sẽ tự động được thực thi** và kết nối về máy A.
- **Lưu ý:** Mọi thao tác trên máy B phải thực hiện **dưới quyền user bình thường**, không có quyền Administrator.

Gợi ý:

Bước 1. Injector quét tiến trình, nếu thấy có WinRAR đang chạy, chèn shellcode vào bộ nhớ của tiến trình, đồng thời tạo một luồng thực thi (thread) mới để chạy shellcode ngay bên trong WinRAR.

Bước 2. Thread vừa được tạo trong WinRAR sẽ kích hoạt kết nối ngược về C2, báo hiệu rằng shellcode đã được thực thi thành công.

Bước 3. Máy chủ C2 tiếp tục gửi thêm payload bổ sung để mở rộng quyền kiểm soát trên WinRAR.

Bước 4. WinRAR nhận đủ payload cần thiết và bắt đầu giao tiếp với C2, tạo điều kiện cho kẻ tấn công kiểm soát hệ thống mà không gây ảnh hưởng đến hoạt động bình thường của ứng dụng.

Dừng lại và suy nghĩ:

- Làm thế nào để shellcode hoạt động mà không làm WinRAR bị lỗi?
- Có cách nào ẩn shellcode để tránh bị Windows Defender phát hiện không?

F. YÊU CẦU & ĐÁNH GIÁ

- Sinh viên tìm hiểu và thực hành theo hướng dẫn, thực hiện **theo nhóm đã đăng ký**.
- Nộp báo cáo kết quả gồm **Code, File được export** và chi tiết những việc (**Report**) mà nhóm đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả (nếu có); giải thích cho quan sát (nếu có).

Báo cáo:

- File **.PDF**. Tập trung vào nội dung, không mô tả lý thuyết.
- Đặt tên theo định dạng: **[Mã lớp]-LabX_MSSV1-MSSV2-MSSV3**.

Ví dụ: [NT230.Q1X.ANTT]-Lab1_2352xxxx-2352yyyy.

- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Bài sao chép, trễ, ... sẽ được xử lý tùy mức độ vi phạm.

HẾT

Chúc các bạn hoàn thành tốt!